Does Financial Instruction Tuning Actually Help?

LLMs in Finance Seminar — Implementation Track

June 25, 2026

CONTENTS

Does Financial Instruction Tuning Actually Help?

AN IMPLEMENTATION REPORT REPRODUCING *OPEN-FINLLMS* ON FINANCIAL SENTIMENT AND NER

LLMs in Finance Seminar — Implementation Track Paper reproduced: *Open-FinLLMs: Open Multimodal Large Language Models for Financial Applications* Task: inference-only reproduction · Compute: Modal T4 (4-bit) · Total spend: **~$1.3**

Executive summary

The *Open-FinLLMs* paper makes a single, testable promise: that **financial instruction tuning** — continuing to train a general LLM on finance instructions — buys you measurably better performance on downstream financial NLP. We tried to reproduce that promise on the two tasks where it should be easiest to see: **sentiment classification** (Financial PhraseBank, FiQA-SA) and **named-entity recognition** (FiNER-ORD, FIN/Alvarado-2015).

We could not get the paper's headline model (`FinLLaMA-instruct`) because the authors un-published it (a 404 — a finding in itself), so we substituted **plutus-8B-instruct**, the same group's current 8B financial-instruct model, and benchmarked it against general 7–8B instruction models (Mistral-7B, Qwen2.5-7B), small in-domain classifiers (FinBERT, FinBERT-tone), a lexicon (VADER), and specialised NER models (GLiNER).

Three findings, each reproduced from the committed predictions:

1. **Financial instruction tuning did not pull ahead — on any of the four datasets.** plutus-8B is mid-pack on sentiment (beaten by Mistral on FPB, by Qwen on FiQA) and is the **single worst LLM on both NER datasets**. The paper's central claim does not replicate on this evaluation.
2. **Prompt choice rivals model choice.** Swapping one prompt template for another moves macro-F1 by up to **16 points** on the same model and dataset — larger than most model-to-model gaps. Any single-prompt LLM "leaderboard" is fragile.
3. **A cheap prompt-ensemble recovers most of that lost robustness.** Majority-vote over four prompt variants beats the *mean* single prompt on **6/6** model×dataset cells, and a cross-validation-weighted vote keeps ~100% answer coverage.

Honest scope: this is a **toy-scale, inference-only** reproduction (300-sentence sentiment subsamples; 98–300-example NER sets; 4-bit quantization; no fine-tuning). The numbers are not meant to rank these models definitively — they are meant to test one claim, carefully, with baselines and an honest discussion of failure. A dedicated **"What we got wrong"** section (§10) documents the missteps.

---

1. Motivation and problem setting

Financial text — earnings releases, analyst notes, filings, headlines — is the raw material of a large part of quantitative finance. Two NLP primitives sit under most finance applications: **sentiment** (is this text good or bad news for the name?) and **entity recognition** (which companies, people, and places does it mention?). If general-purpose LLMs already do these well, the finance industry can use them off the shelf. If a *financially tuned* LLM does them meaningfully better, that tuning is worth its cost. *Open-FinLLMs* argues the latter. Because that claim drives real build-vs-buy decisions, it is worth reproducing rather than taking on faith — exactly the "validate on a held-out set; do not trust the model's marketing" discipline.

2. Background: the paper, condensed

Open-FinLLMs introduces a family of open financial LLMs (FinLLaMA, and a multimodal FinLLaVA) built by continually pre-training and instruction-tuning LLaMA on a large financial corpus, then evaluates them across financial NLP benchmarks — sentiment, NER, classification, QA, and multimodal tasks — reporting that the financial models beat their general-purpose bases and rival much larger commercial models on several tasks.

We reproduce the slice that is feasible on a single T4 GPU and that most directly tests the tuning claim:

- **Sentiment (3-class: positive / neutral / negative).**
    - **Financial PhraseBank (FPB)** — finance news sentences, each labelled by 5–8 finance-trained annotators. The dataset ships **four nested inter-annotator-agreement subsets**: `AllAgree` (100% / unanimous, 2,264 sentences), `75Agree` (≥75%, 3,453), `66Agree` (≥66%, 4,217), `50Agree` (≥50%, 4,846). We report on the **75%-agreement split** (690 test sentences, 20% seed-42 hold-out) — the conventional FPB benchmark setting, balancing label quality against sample size — and additionally validate on the **100%-agreement gold** in §6.4.
    - **FiQA-SA** — microblog/news financial sentiment with a continuous score in [-1, 1], bucketed to 3 classes with a ±0.10 neutral band (1,173 test items).

- **NER (entity types PER / ORG / LOC).**
  - **FiNER-ORD** — financial NER over news (1,075 test sentences; we subsample).
  - **FIN / Alvarado-2015** — the dataset behind the paper's Table 7 NER column (the public PIXIU `flare-ner` subset is 98 examples).

Baselines that matter for the claim:

- **FinBERT** (`ProsusAI/finbert`) and **FinBERT-tone** (`yiyanghkust/finbert-tone`) — 110M finance-tuned classifiers; the conventional, cheap finance sentiment tools.
- **VADER** — a rule/lexicon sentiment model with no learning; the "is this better than a dictionary?" floor.
- **GLiNER** (small/large) — specialised zero-shot NER models.

3. Objective — exactly what we reproduced, adapted, and simplified

Choice	What we did	Why
Model under test	`plutus-8B-instruct` substituted for `FinLLaMA-instruct`	FinLLaMA-instruct returns **404** (un-published). plutus is the same group's current 8B financial-instruct model. Comparison is *indicative*, not identical.
Mode	Inference only — **no fine-tuning**	Tests the *released* tuning, on a T4 budget.
Sentiment scale	300-sentence balanced subsample per LLM run; full test set for the cheap baselines	Keeps 24 LLM runs inside a < $1 GPU budget; baselines are free so they run on everything.
NER scale	FiNER-ORD 200–300 subsample; FIN = the full public 98-example set	Same budget logic; FIN is small because only the PIXIU subset is public.
Precision	4-bit (bitsandbytes) for every 7–8B model	Two 8B models will not coexist on a 16 GB T4 otherwise.
Prompts	Templates A (minimal), B (analyst definition list), C (market-reaction) × {0, 3}-shot	To *measure* prompt sensitivity rather than assume a single prompt is representative.

Models we **could not** evaluate (documented, not silently dropped): `FinLLaMA-instruct` (404), `LLaMA-3.1-8B-Instruct` (gated, no access), `FinMA-7B-full` (gated, 403), `Qwen3-8B` initially (needed a newer `transformers`; later run on the NER track).

4. System design

The pipeline is a small, testable library (`src/`) driven by scripts; the same shape is reused for both tasks.

```
                          data_loader           prompts           runner
parser             evaluation
 HF datasets  ──▶  unified Sample dicts  ──▶  build_prompt  ──▶  LLMRunner (Modal T4)
──▶  parse(raw)  ──▶  compute_metrics
   FPB / FiQA     {id,text,label,...}        (LLMs only)       FinBERTRunner/VADER
(LLMs only)      acc / F1 / coverage
   FiNER / FIN                                                 GLiNER (local)
confusion matrix
```

Key design invariants:

- **One unified `Sample` schema** for every dataset: `{id, text, label, dataset, split}`, labels in `{positive, neutral, negative}`. FiQA's continuous score is bucketed via the neutral band.
- **Runner split.** `LLMRunner.generate()` returns *raw text* + latency; parsing happens outside it. `FinBERTRunner`/`VADERRunner`/GLiNER return canonical labels directly (no parser needed, so they are always 100% coverage).
- **Three deliberately diverse prompts** so the ensemble members disagree in useful ways: **A** = "Classify the sentiment … positive/negative/neutral"; **B** = "You are a financial analyst … " with an explicit definition list; **C** = a market-reaction framing ("how would this move an investor's outlook — bullish / bearish / neutral").
- **Software stack.** `transformers` + `bitsandbytes` (4-bit) on a **Modal** T4 container with a hard **$7 budget guard** and a per-run cost ledger; HF weights cached in a Modal volume. Checkpoint/resume via per-run `meta.json` / `predictions.jsonl` / `progress.json`.

Major simplifications: 4-bit (not fp16); subsampled test sets for LLMs; zero-/few-shot only (no fine-tuning); a single prompt template × 0-shot for NER.

5. Experimental design

- **Metrics.** Accuracy, **macro-F1** (the headline — it does not let a dominant class hide errors), weighted-F1, per-class P/R/F1, and a 3×3 confusion matrix. For NER: entity-level strict micro/macro-F1 and per-type F1 (PER/LOC/ORG); FIN also reports a "partial" match.
- **Coverage, and why we never force a label.** When an LLM's output cannot be parsed to a label, the prediction is recorded as `parse_ok=False` and **excluded from accuracy/F1**, then reported separately as **coverage**. Forcing unparseable outputs to "neutral" would silently inflate a model that hedges — so we refuse to. This makes coverage a first-class result (a model can win on F1 but lose on coverage, or vice-versa).
- **Determinism.** `seed = 42` everywhere; greedy decoding (`temperature = 0`); balanced few-shot sampling from the FPB train pool.
- **Uncertainty.** Headline macro-F1s carry **95% bootstrap confidence intervals** (2,000 resamples of the per-example predictions; `bootstrap_ci.py`), so the reader can see which gaps are real and which are subsample noise.
- **Baselines.** Every sentiment claim is measured against FinBERT, FinBERT-tone, and VADER on the *same* items; every NER claim against GLiNER-small/large.
- **Leakage and contamination — the honest caveats.** (i) These benchmarks are old and public; any of these models may have seen FPB/FiQA/FIN during pre-training, which would *inflate* all of them — we cannot rule it out. (ii) For NER we use **strict** span matching, which is harsh (see §8). (iii) plutus is a *substitute* for the paper's model, so the gap to the paper's numbers mixes a real effect with a model-identity difference.

6. Results — sentiment

Headline (best macro-F1 per model × dataset):

Model	FPB $F1_{m}$	FPB Acc	FiQA $F1_{m}$	FiQA Acc
FinBERT (110M, in-domain)	**0.925**	0.935	0.482	0.498
FinBERT-tone (110M)	0.855	0.881	0.396	0.389
Mistral-7B-Instruct-v0.3 (general)	**0.890**	0.903	0.599	0.620
Qwen2.5-7B-Instruct (general)	0.832	0.860	**0.673**	0.717
plutus-8B-instruct (financial)	0.829	0.851	0.597	0.657
VADER (lexicon)	0.469	0.554	0.386	0.423

Best macro-F1 by model and dataset

> **Significance.** 95% bootstrap CIs (2,000 resamples; `bootstrap_ci.py`) are **≈±0.04–0.05** for the 300-sentence LLM runs and **±0.02–0.03** for the full-test baselines. Differences smaller than that are *not* significant — so on FPB plutus≈Qwen (CIs [0.776, 0.873] vs [0.784, 0.878]) are statistically tied, and FinBERT's FPB lead over Mistral is within noise. Bold marks the highest point estimate, not a proven winner. Full CIs in `results/summary/bootstrap_ci.csv`.

Interpretation. The financial-tuned model is *never the best LLM*. On FPB the order is FinBERT ≳ Mistral > FinBERT-tone > plutus ≈ Qwen; on FiQA it is Qwen > Mistral ≈ plutus > FinBERT. Two things follow:

- **Financial tuning didn't pull ahead.** plutus-8B is solid but mid-pack. If the tuning added a robust downstream advantage, plutus should top at least one dataset. It tops neither.
- **Small in-domain models don't generalise.** FinBERT tops FPB (0.925; though within noise of Mistral) but **collapses on FiQA (0.482)** — a 44-point drop, far outside any CI, on a different financial register (microblog/news). The cheap specialist is the best tool *only* on the distribution it was trained for. This is a genuinely useful production lesson.

6.1 PROMPT SENSITIVITY RIVALS MODEL CHOICE

The same model, same dataset, same 0-shot setting, only the **prompt** changed:

Model · dataset	Template A ($F1_m$)	Template B ($F1_m$)	Gap
Mistral-7B · FPB	0.803	0.689	**11.4 pt**
plutus-8B · FiQA	0.597	0.432	**16.4 pt** (26 pt accuracy)
Qwen2.5-7B · FiQA	0.579	0.673	9.4 pt (B wins)

Coverage and prompt behaviour across templates and shots

These swings are **as large as, or larger than, the differences between models.** A benchmark that reports one prompt per model is measuring the prompt as much as the model. This is the methodological hole that motivated the ensemble (§6.3).

6.2 FEW-SHOT HELP IS NOT FREE

Effect of 3-shot vs 0-shot

Few-shot examples **help on FPB** (Mistral A: 0.803 → 0.890) but **hurt on FiQA** (Mistral A: 0.599 → 0.576; plutus A: 0.597 → 0.497) — the FPB-sampled exemplars are off-distribution for FiQA's microblog text and drag performance down. "Add few-shot" is not a universal win; it depends on whether the exemplars match the target register.

6.3 A PROMPT-ENSEMBLE BUYS BACK ROBUSTNESS (METHODOLOGICAL CONTRIBUTION)

If a single prompt is unreliable, vote over several. We build a 4-member ensemble (templates A/B × {0,3}-shot) and aggregate by majority vote, with three tie-break policies and a cross-validation-**weighted** vote (member weights estimated by 5-fold CV on the test predictions — leakage-free for weighting).

Ensemble vs single-prompt, per model and dataset

Aggregator	Mean $F1_m$ (6 cells)	Mean coverage	Beats mean-single	Beats best-single
Unweighted majority (abstain on tie)	**0.699**	0.887	**6 / 6**	0 / 6
CV-weighted vote	0.694	**0.996**	**6 / 6**	1 / 6

Coverage–F1 trade-off across aggregation policies

Interpretation. The ensemble reliably beats the *expected* single prompt (the one you would pick blind) on every cell — per-cell gains of **+0.02 to +0.08 F1** — and never collapses. It does not beat the *oracle best* single prompt (the one you would only know in hindsight), which is the honest ceiling. The **CV-weighted** vote is the deployable choice: it keeps ~100% coverage (no abstentions) at almost the same F1. (Caveat: CV-weighting needs labels to estimate member weights, so it is the *best-with-a-validation-set* option, not a label-free one — unweighted majority is the truly zero-label default.) Net: ensembling converts "pick the right prompt and hope" into a stable default.

6.4 GOLD-LABEL QUALITY: 75% VS 100% ANNOTATOR AGREEMENT

A model is only as trustworthy as the labels it is scored against, so we re-ran the **full pipeline on FPB's 100%-agreement (AllAgree) split** — the same models, templates, and shots as the headline, but on the 452-sentence unanimous-label test set (300-sentence subsample for the LLMs; baselines on all 452). This is a proper re-evaluation, not a re-weighting (a separate 14-cell run on Modal; `run_allagree.py`).

FPB macro-F1 at 75% vs 100% annotator agreement

Model (best config)	**$F1_m$ @ 75%-agree**	**$F1_m$ @ 100%-agree**	**Δ**
FinBERT	0.925	**0.977**	+0.052
Mistral-7B (A/3-shot)	0.890	**0.937**	+0.047
Qwen2.5-7B (A/3-shot)	0.832	**0.928**	+0.096
plutus-8B (A/3-shot, financial)	0.829	**0.851**	**+0.022**
VADER (lexicon, no learning)	0.469	0.483	+0.014

(FinBERT-tone is absent here: its checkpoint config no longer loads under the required `transformers` version — a reproducibility failure we log in §10.)

Interpretation. Three things, all on-thesis:

1. **Gold-label quality drives the score.** Every learned model improves on the unanimous gold — the general LLMs and FinBERT by ~5–10 F1 points (Qwen +9.6, FinBERT +5.2, Mistral +4.7) — because most of their "errors" on the 75% set were on the sentences humans themselves disagreed about, a textbook demonstration that label quality, not just the model, sets the ceiling.
2. **VADER is the control.** A lexicon that does no learning barely moves (+0.014), confirming the gains are real model behaviour on cleaner labels, not an artefact of an easier subset.
3. **The financial model gains the *least* — and falls *further* behind.** plutus-8B improves only +0.022, the smallest of any learned model. On the cleanest gold the gap actually **widens**: plutus (0.851) now trails Mistral (0.937) by 8.6 points and Qwen (0.928) by 7.7 — versus a 6-point / 0.3-point gap on the 75% set. Its remaining errors are on *unambiguous* sentences, so removing label noise helps it least. The financial-tuning conclusion does not just survive the stricter gold — it gets sharper. And here the gap is **statistically real**: on AllAgree plutus's 95% CI [0.806, 0.894] does not overlap Mistral's [0.908, 0.964] or Qwen's [0.896, 0.955], whereas on the 75% set the top models' CIs all overlapped.

(A no-GPU cross-check that instead *filters* the existing 75%-agree predictions to their unanimous-agreement subset — `eval_allagree.py` — gives the same ranking and the same direction, an independent confirmation.)

7. Results — NER

NER: plutus-8B is worst on both datasets

FiNER-ORD (entity strict micro-F1; all models on the same 300-sentence test subsample):

Model	n	strict-F1	coverage
Qwen2.5-7B-Instruct	300	**0.628**	1.00
Qwen3-8B	300	0.603	1.00
GLiNER-small (specialist)	300	0.594	1.00
GLiNER-large (specialist)	300	0.561	1.00
Mistral-7B-Instruct-v0.3	300	0.535	1.00
Qwen3-4B	300	0.494	1.00
plutus-8B-instruct (financial)	300	**0.365**	0.853

FIN / Alvarado-2015 (entity micro-F1, 98 examples):

Model	micro-F1	coverage	paper analog (Table 7)
Qwen2.5-7B-Instruct	**0.160**	0.84	—
Mistral-7B-Instruct-v0.3	0.153	0.94	Mistral-v0.1: 0.00
plutus-8B-instruct (financial)	**0.107**	1.00	FinLLaMA-instruct: **0.57**

Interpretation.

- **The tuning claim fails again, more clearly.** plutus-8B is the **worst LLM on both** NER datasets — and on FiNER-ORD it is the only model that also drops coverage (0.853). The general Qwen2.5 leads both. GLiNER (a tiny specialist) beats Mistral and the smaller Qwen3-4B on FiNER-ORD and loses only to the two strongest general models (Qwen2.5, Qwen3-8B) — a reminder that a purpose-built NER model is competitive with much larger chat models at a fraction of the size.
- **No open model reaches half the paper's FIN score** (best 0.160 vs the paper's 0.57 for FinLLaMA-instruct). Most of that gap is **methodological**, not a quality collapse: strict span matching, 4-bit, a 98-example set, and a substitute model (see §8–§9). The paper's "Entity F1" likely normalises case/punctuation before matching; we did not.
- **A one-version bump moves NER 15 points.** The paper lists Mistral-**v0.1** at 0.00 (it refused/unparseable). Mistral-**v0.3** follows the format and reaches 0.153. Minor version identity matters enormously for these comparisons — a reproducibility warning in its own right.

8. Error analysis

8.1 SENTIMENT — WHERE PLUTUS-8B MISSES (30 HAND-TAGGED FPB ERRORS)

Category	Count	What it is
missed_positive_cue	**15 (50%)**	Mild / forward-looking positives read as neutral (neutral-bias)
numerical_reasoning	8 (27%)	Needs comparing figures: “loss narrowed 3.7→1.8mn”, “profit 17.7 vs 17.6”
factual_neutral_misclassed	6 (20%)	A neutral fact (appointment, M&A, delisting, restructuring) read as pos/neg
ambiguous / out-of-domain	1 (3%)	Genuinely unclear sentence

Confusion matrices across models and datasets

The error profile is dominated by a **conservative neutral-bias**: half the misses are positives that plutus down-graded to neutral (“plans to expand internationally”, “renewed its contract with Fujitsu”, “expects net sales to increase”). A further quarter are **numerical-reasoning** failures — the model sees the word “loss” and answers negative even when the loss *narrowed* year-on-year (genuinely positive). Only one of thirty errors is truly ambiguous; these are mostly *real* mistakes, not label noise. The per-class confusion grid confirms the neutral-column is where probability mass leaks.

8.2 NER — STRICT MATCHING PUNISHES A WELL-BEHAVED MODEL

plutus on FIN has **perfect format compliance (0 parse failures)** yet the lowest F1. Reading its outputs, the misses are overwhelmingly **span-boundary disagreements** — “Evergreen Solar Inc.” vs gold “Evergreen Solar Inc” (trailing period), and all-caps duplicates the model merges into one span. The model is not hallucinating entities; it is losing a strict string match. This is exactly the case where a **normalised-match** metric would close much of the gap — and a caution against reading a single strict-F1 number as “the model can’t do NER”.

9. Critical assessment and limitations

- **The model is a substitute.** plutus-8B ≠ FinLLaMA-instruct. Our “financial tuning didn’t help” is strictly a statement about plutus; it is *consistent with* the paper’s model being un-reproducible, but it is not a direct refutation.

- **Statistical power is thin.** 300-sentence sentiment subsamples and 98–300 NER items mean confidence intervals are wide; small F1 gaps (e.g. plutus vs Qwen on FPB) should not be over-read.
- **4-bit, not fp16.** Quantization can cost a few points; the paper likely ran full precision. This handicaps *all* our LLMs equally but widens the gap to the paper.
- **Strict NER matching** understates real performance (§8.2).
- **Possible benchmark contamination.** FPB/FiQA/FIN are old and public; pre-training exposure would inflate every model and is unmeasurable here.
- **Coverage is not free F1.** plutus's FiQA template-B run answers only when confident; we report its F1 on what it answered and its coverage separately, rather than papering over the abstentions.

What I would distrust most about these numbers: the absolute NER F1 (strict matching + tiny FIN set) and any sentiment gap inside the bootstrap CIs — for the 300-sample LLM runs that is **~±0.045**, so two-model differences under ~0.06–0.09 F1 are not significant (e.g. plutus vs Qwen on FPB). What I would trust: the *direction* — financial tuning not leading on any of four datasets, prompt variance being large, and plutus falling *significantly* behind on the 100%-agreement gold — because these hold across datasets, seeds, aggregation policies, and now CIs.

10. What we got wrong during the research

An honest log of the missteps — these cost the most time and are the most useful to the next person:

1. **Assumed the paper's model would be downloadable.** `FinLLaMA-instruct` was un-published mid-project (404). We lost time before accepting it and substituting plutus-8B. *Lesson:* pin and cache the exact artifact on day one; treat "available on HF" as a risk, not a given.
2. **Trusted the default `transformers` version.** plutus returned **empty strings** on raw prompts until we (a) bumped `transformers` 4.44→4.50 for its tokenizer and (b) enabled the **chat template** (`apply_chat_template`) — instruction-tuned models need their chat format, not a bare prompt. Qwen3-8B was blocked entirely until a newer `transformers`. *Lesson:* version-and-chat-format mismatches look like "the model is bad" but are harness bugs.
3. **Initially read plutus's low NER score as incompetence.** It was **strict span-matching** (trailing periods, case) against a model with *perfect* format compliance. We almost reported a wrong conclusion before inspecting raw outputs. *Lesson:* always read the raw misses before believing a metric.

4. **Compared against the paper's Mistral-v0.1 as if versions were interchangeable.** v0.1 → v0.3 swings NER by 15 points. *Lesson:* record exact model revisions.
5. **Briefly committed a real `HF_TOKEN` to a local commit before redacting it.** *Lesson:* secrets never touch the repo; rotate immediately if they do (we did).
6. **Let results scatter across branches.** The complete sentiment matrix, the prompt-ensemble, the FiNER-ORD NER, and the FIN/Alvarado NER each lived on a *different* branch/worktree; the FIN reproduction was nearly lost. This report re-consolidated them and **re-ran every aggregation to verify the numbers**. *Lesson:* one place for results, and re-aggregate from raw predictions before writing anything down.
7. **FinBERT-tone stopped loading.** Our earlier `finbert-tone` baseline (in the 75%-agree table) no longer instantiates under the `transformers` version the rest of the stack now requires — its `config.json` lacks a `model_type` the newer loader demands ("Unrecognized model"). We caught it when the AllAgree run skipped it, and report the AllAgree comparison without it. *Lesson:* a working baseline can silently rot across a dependency bump; pin per-model environments or re-verify every baseline each run.
8. **A test-suite environment wrinkle.** The full `pytest` run shows 11 spurious failures in `test_evaluation.py` from a `scipy`/`array_api_compat` caching quirk triggered by test ordering — every test **passes in isolation**, and the evaluation code is the same code that produced the (verified) tables. *Lesson:* distinguish environment artifacts from logic regressions before "fixing" them.

11. Lessons learned

- **Reproducibility is the real finding.** Two of our headline results are *meta*-results: a paper model can vanish, and a minor version bump can swing a benchmark by 15 points. LLM benchmark numbers are far less portable than they look.
- **Prompt ≥ model, often.** Report a distribution over prompts, or ensemble — never a single prompt — if you want a claim that survives a re-run.
- **Coverage belongs next to accuracy.** A model that hedges should not be rewarded by a force-to-neutral default.
- **Gold-label quality drives the score — and re-ranks robustness.** Re-running on FPB's unanimous-agreement gold lifts every learned model 5–10 F1 points (VADER flat), and the financial-tuned model gains the *least*, widening its deficit. Always report which agreement level you used and, ideally, validate on the cleanest subset.
- **Right tool for the task.** FinBERT for in-distribution sentiment; GLiNER for NER; general chat LLMs as flexible-but-not-dominant generalists. The financial-tuned 8B did not earn a default slot anywhere in our evaluation.

What we'd do with another month: add a normalised-match NER metric and re-run (likely closes much of the FIN gap); get access to `FinMA-7B-full` and the real `FinLLaMA-instruct` if re-published; run fp16 on a bigger GPU to remove the quantization confound; complete the prompt-ensemble's template-C members (E3/E5); and bootstrap confidence intervals on every F1.

12. Conclusion

1. On four financial datasets, **financial instruction tuning (plutus-8B) never led** — mid-pack on sentiment, worst on NER. Strictly, this tests the claim on a *proxy* (plutus stands in for the unpublished FinLLaMA-instruct), so it is evidence against the general "financial tuning helps" thesis rather than a direct refutation of the paper's specific model. On this evaluation the claim did not reproduce.
2. **Prompt variance is first-order**, and a cheap prompt-ensemble converts it from a liability into a stable default.
3. The most transferable lessons are about **reproducibility** — disappearing artifacts, version sensitivity, and the discipline of re-aggregating from raw predictions — which matter more than any single F1 in the tables.

Appendix A — Reproducibility

What ships, and what reproduces from it. The committed summary tables and confusion matrices (`results/summary/*.csv`, `results/summary/confusions*/`) are in the deliverable; the raw per-run `results/predictions/` is **not** (it is bulky model output). So the **figures** rebuild from the shipped CSVs with no GPU, but **re-aggregating the tables** (step 2) needs `results/predictions/`, which means either the full repo checkout or a re-run of inference (step 4).

```
# 1. Environment
python -m venv .venv && . .venv/Scripts/activate
pip install -r requirements.txt

# 2. Re-aggregate tables — needs results/predictions/ (full repo, not the code zip)
python scripts/aggregate.py           # -> results/summary/final_table.csv (sentiment)
python scripts/aggregate_ensemble.py  # -> results/summary/ensemble_table.csv
python scripts/aggregate_ner.py       # -> results/summary_ner/final_table_ner.csv
(FiNER-ORD)
python scripts/summarize_ner.py       # -> results/summary/ner_table.csv (FIN/Alvarado)
python scripts/bootstrap_ci.py        # -> results/summary/bootstrap_ci.csv (95% CIs;
needs predictions)
python scripts/eval_allagree.py       # no-GPU cross-check (needs predictions): filter
75%-agree to unanimous

# 3. Rebuild every figure — works from the committed CSVs alone (no GPU, no predictions)
python scripts/make_figures.py
python scripts/make_ensemble_figures.py
python scripts/make_ner_figure.py
python scripts/make_agreement_figure.py # -> agreement_comparison.csv + figure (75% vs
100%)
python scripts/tag_errors.py           # -> error taxonomy in focal_error_sample.csv
(needs focal CSV, shipped)

# 4. (Optional) re-run inference on Modal T4 — costs ~$1, needs HF_TOKEN
python scripts/run_llm_matrix.py      # sentiment matrix (budget-guarded)
python scripts/run_allagree.py        # 100%-agreement (AllAgree) matrix ->
allagree_table.csv
python scripts/run_ner_matrix.py      # FIN/Alvarado NER
```

Environment notes: `transformers >= 4.50` and the chat template are required for plutus-8B; Qwen3 needs `>= 4.51`. Set `HF_TOKEN` for gated weights. Seed = 42. Total compute for the whole study: **~$1.3** of Modal T4 time (≈ 130 GPU-minutes, across the sentiment matrix, the 100%-agreement matrix, and both NER tracks).

Appendix B — Datasets, models, and artifacts

Component	Source
FPB	`financial_phrasebank` (75%-agree split), 690 test
FiQA-SA	`TheFinAI/fiqa-sentiment-classification`, 1,173 test, ±0.10 neutral band
FiNER-ORD	financial NER, 1,075 test (subsampled)
FIN / Alvarado-2015	PIXIU `TheFinAI/flare-ner`, 98 test
Model under test	`TheFinAI/plutus-8B-instruct` (substitute for `FinLLaMA-instruct`)
General LLMs	`mistralai/Mistral-7B-Instruct-v0.3`, `Qwen/Qwen2.5-7B-Instruct`, `Qwen/Qwen3-8B`, `Qwen/Qwen3-4B-Instruct-2507`
Classifiers / lexicon	`ProsusAI/finbert`, `yiyanghkust/finbert-tone`, VADER
NER specialists	`urchade/gliner_small-v2.1`, `urchade/gliner_large-v2.1`

References

- *Open-FinLLMs: Open Multimodal Large Language Models for Financial Applications.*
- Malo et al., *Good Debt or Bad Debt: Financial PhraseBank.*
- *FiQA 2018* — Financial Opinion Mining and Question Answering.
- Shah et al., *FiNER-ORD: Financial NER*; Alvarado et al., 2015 (FIN).
- Araci, *FinBERT*; Yang et al., *FinBERT-tone*; Hutto & Gilbert, *VADER*.
- Zaratiana et al., *GLiNER*.